



Desayuno Asíncrono

"No se trata de correr más rápido, sino de saber qué carreras correr en paralelo."

Simulación de un desayuno con 7 acciones con tiempos conocidos y un presupuesto de **500 ms** antes de que el café se enfríe.



Cómo ejecutar

```
dotnet run
```

Proyecto de consola en C# (.NET 10). Se implementan y comparan 5 enfoques:

1. Ejecución **secuencial** (síncrona).
2. Ejecución con **async/await secuencial** (sin paralelismo).
3. Ejecución **paralela** con **Task.WhenAll** (mejor rendimiento).
4. Ejecución **optimizada por camino crítico** (cadenas de tareas).
5. Las anteriores con **timeout de 500 ms** (**CancellationTokenSource**).



Las 7 acciones

#	Acción	Tiempo	Dependencia
1	Hacer café	200 ms	ninguna
2	Calentar sartén	200 ms	ninguna
3	Freír huevos	300 ms	sartén caliente (2)
4	Freír bacon	300 ms	sartén caliente (2)
5	Tostar pan	200 ms	ninguna
6	Untar mantequilla	100 ms	pan tostado (5)
7	Hacer zumo	200 ms	ninguna



Tabla de tiempos

Resultados reales (media de varias ejecuciones en mi máquina):

Solución	Tiempo	¿Cumple 500 ms?
1. Síncrono	~1550 ms	✗
2. Async/await secuencial	~1560 ms	✗
3. Async/await paralelo (mejor rendimiento)	~514 ms	✗ (roza el límite)
4. Async/await optimizado (camino crítico)	~515 ms	✗ (roza el límite)

Con timeout de 500 ms

Solución	Tiempo de cancelación	Resultado
1. Síncrono	~730 ms	✗ Cancelado a tiempo
2. Async/await secuencial	~510 ms	✗ Cancelado a tiempo
3. Async/await paralelo	~511 ms	✗ Cancelado a tiempo
4. Async/await optimizado	~501 ms	✗ Cancelado a tiempo

☕ "¡El café se ha enfriado! Los huevos y tostadas con café frío no tienen gracia..."

En todas las ejecuciones se mostró el mensaje de café frío: **ninguna solución logra entregar el desayuno dentro de 500 ms.**

💡 Preguntas y reflexiones

¿Qué diferencias has observado entre las 5 soluciones?

La solución **síncrona** y la de **async/await secuencial** tardan lo mismo (~1550-1560 ms): la suma de las 7 acciones. La clave es que **async/await por sí solo no aporta paralelismo**: cada **await** espera a que termine la acción actual antes de lanzar la siguiente, así que el tiempo es idéntico al síncrono (la única diferencia es que el hilo principal no queda bloqueado durante las esperas).

La solución **paralela** (**Task.WhenAll**) reduce el tiempo a **~514 ms** (mejora de ~3×) porque lanza a la vez las tareas independientes. La versión **optimizada** (por camino crítico) obtiene **~515 ms**: no mejora a la paralela porque el cuello de botella es la cadena sartén→huevos/bacon (500 ms), que ninguna reordenación puede reducir.

Con **timeout de 500 ms** las cuatro se cancelan y muestran el café frío. El paralelo y el optimizado son los que más se acercan (~501-514 ms), pero se cancelan porque su tiempo natural es ligeramente superior al límite.

¿Qué acciones se pueden ejecutar a la vez y cuáles no? ¿Por qué?

A la vez (independientes): café, zumo, calentar la sartén y tostar el pan. No dependen de ninguna otra acción.

Tras un punto de sincronización: huevos y bacon se pueden lanzar juntos en cuanto la sartén está caliente (dependen de la acción 2, pero no entre sí).

En secuencia (dependientes):

- La **mantequilla** necesita el pan tostado: no puede untarse antes de que termine el tostado.
- Los **huevos y el bacon** necesitan la sartén caliente: no pueden freírse mientras se calienta.

Una dependencia es una relación de "antes de" (precedencia). Si se viola, el resultado es incorrecto (tostada sin tostar o huevos crudos), por eso estas acciones forman el **camino crítico**: sartén (200) → huevos (300) = 500 ms, que es el mínimo teórico del desayuno.

¿Qué ha pasado con cada solución cuando introduces el timeout?

Las cuatro soluciones se **cancelan** y muestran el mensaje de café frío, porque ninguna termina dentro de 500 ms:

- **Síncrono** (~730 ms): se canceló tarde porque `Thread.Sleep` **no puede interrumpirse a mitad de una acción**. El token solo se comprueba entre acciones, así que el hilo termina la acción en curso antes de cancelarse. Es un buen ejemplo de código bloqueante que no coopera con la cancelación.
- **Async/await secuencial** (~510 ms): la cancelación es inmediata porque `Task.Delay(ms, token)` respeta el token y lanza `OperationCanceledException` justo a los 500 ms.
- **Async paralelo** (~511 ms): se cancela al borde del límite. Su mejor caso teórico es 500 ms exactos, y con la sobrecarga del planificador se va a ~514 ms, así que **queda justo fuera del presupuesto**.
- **Async optimizado** (~501 ms): igual que el paralelo; el camino crítico (500 ms) hace imposible terminar por debajo del límite.

¿El enfoque con mejor rendimiento es también el más seguro? ¿Por qué?

No. El enfoque paralelo es el más rápido pero también el más arriesgado:

- **No deja margen:** ronda el límite de 500 ms; cualquier fluctuación del sistema (sobrecarga, planificador, GC) lo hace fallar. En todas mis ejecuciones se canceló.
- **Añade complejidad de concurrencia:** hay que respetar las dependencias y esperar a los puntos de sincronización; un error de orden produce un resultado incorrecto (mantequilla sin pan, huevos con la sartén fría).
- **Simplificar la lógica:** con `Task.WhenAll` las excepciones y tiempos de varias tareas se mezclan, más difícil de depurar que una secuencia simple.

El mejor rendimiento sin **margen de seguridad** es frágil: rendimiento extremo y robustez suelen estar en tensión.

¿Merece la pena complicarse con paralelismo o con mecanismos de control de tiempo?

Justifica tu respuesta.

Sí, merece la pena.

- **Paralelismo:** pasar de ~1560 ms a ~514 ms es una mejora de ~3× sin cambiar la naturaleza del trabajo, solo ejecutando a la vez lo que no depende entre sí. El coste es bajo cuando se gestionan bien las dependencias con un par de `await` y `Task.WhenAll`.
- **Mecanismos de control de tiempo:** la concurrencia no es determinista; no sabes con exactitud cuándo acabarán las tareas. `CancellationTokenSource` permite definir un presupuesto y cancelar si se supera, en vez de esperar eternamente.

Ahora bien, este ejemplo también muestra el límite de complicarse: el mejor paralelismo posible en este desayuno es 500 ms, y el presupuesto es 500 ms. **Por mucho que se optimice el orden de ejecución, no se puede ganar si el camino crítico iguala al presupuesto.** En el mundo real la lección sería: o se reduce el tiempo de las acciones críticas, o se aumenta el presupuesto, o se acepta que a veces el café saldrá frío.

Conclusión

"Ninguna solución cumple el límite de 500 ms". El paralelismo ofrece la mayor mejora y el timeout aporta la seguridad de no esperar para siempre, pero el **mínimo teórico está limitado por el camino crítico** (sartén + huevos = 500 ms), por lo que el desayuno siempre se queda al borde del café frío.

Enlaces de Interés y Documentación

-  **Repositorio en GitHub:** [DesayunoSincrono-Asincrono](#)